

How a Unified Development Toolchain Supports Automated Testing Consistency in a TypeScript Full-Stack Monorepo: A Case Study of Circles

Aldiyar Seylkanov

*Bachelor Degree Student, School of Software Engineering
Astana IT University (AITU)*

Astana, Kazakhstan

(email)

(orcid)

Alexandr Tyulkov

*Master Degree Student, School of Software Engineering
Astana IT University (AITU)*

Astana, Kazakhstan

widesehl@gmail.com

0009-0009-6422-0559

Abstract—TypeScript monorepos often use different tools for bundling, testing, linting, and formatting across packages. This fragmentation causes the same test code to behave differently depending on which package runs it. This paper examines how adopting a unified development toolchain addresses this inconsistency. The Circles project, a full-stack TypeScript monorepo, is used as a case study. Before unification, each package configured its own test runner and transform pipeline. After adopting Vite+, all packages share a single tool entry point, one module resolver, and one transform pipeline. The paper describes the toolchain architecture, identifies the sources of inconsistency in the fragmented setup, and evaluates the outcome using configuration surface size, test environment uniformity, and CI pipeline structure. Results show that unification eliminates the main categories of cross-package test inconsistency without requiring per-package configuration.

Index Terms—monorepo, TypeScript, unified toolchain, automated testing, Vite, software engineering

I. INTRODUCTION

TypeScript monorepos often combine frontend applications, backend services, and shared packages in one repository. This structure simplifies code sharing, but it also increases toolchain complexity. Different packages may use different test runners, module resolvers, transform steps, and command interfaces. As a result, the same test logic can behave differently across packages even when the application code is correct. In this setting, inconsistency comes from the execution environment rather than from the tests themselves.

This problem is important in full-stack repositories because packages are not isolated in practice. Shared types, utility modules, and common build assumptions move across package boundaries. When each package defines its own testing stack, small differences in resolution rules, TypeScript transforms, or runtime defaults can produce conflicting results. A test may pass in one package and fail in another because the surrounding toolchain is different. The main engineering issue is therefore configuration fragmentation.

This paper examines that issue through the Circles project, a TypeScript full-stack monorepo that contains a React frontend, a Hono backend, and shared packages. The study

focuses on the transition from a fragmented per-package testing setup to a unified toolchain based on Vite+. The analysis is guided by the following research questions:

Research Question 1 (RQ1) How does per-package toolchain fragmentation create inconsistent test behavior across packages in a TypeScript monorepo?

Research Question 2 (RQ2) To what extent does a unified toolchain reduce the configuration surface required to run tests across the monorepo?

Research Question 3 (RQ3) Does sharing one CLI entry point, one resolver, and one transform pipeline produce a more uniform testing and CI execution model in Circles?

The paper makes three contributions. First, it defines toolchain fragmentation as an environment-level source of testing inconsistency in TypeScript monorepos. Second, it describes how a unified toolchain changes the configuration model in Circles. Third, it evaluates the effect of that change using configuration surface size, test environment uniformity, and CI pipeline structure. The remainder of the paper is organized as follows. Section II reviews related work. Section III describes the Circles monorepo and the unified toolchain. Section IV presents the evaluation. Section V discusses limitations and generalizability. Section VI concludes the paper.

II. RELATED WORK

Prior work relevant to this study falls into two main areas: empirical studies of software quality and fault patterns in JavaScript and TypeScript projects, and studies of automated testing in continuous integration environments. These papers explain why reliability problems persist in large TypeScript systems and why automation infrastructure matters, but they do not directly examine unified test environments inside a monorepo.

A. TypeScript Quality and Fault Profiles

Bogner and Merkel compared 604 JavaScript and TypeScript GitHub projects across code quality, understandability, bug proneness, and bug resolution time [1]. Their results suggest that TypeScript projects have better code quality and understandability, but they do not show significantly

lower bug proneness or faster bug resolution. This is relevant because it limits a common explanation of quality improvement. TypeScript may improve some properties of code, yet language choice alone does not remove the broader conditions that produce bugs during development and testing.

Tang, Alimadadi, and Sumner move closer to the present paper by analyzing 633 bug reports from 16 TypeScript repositories and constructing a taxonomy of fault categories [2]. Their main finding is that tooling and configuration faults, API misuse, and asynchronous error handling are more prominent than pure logic or syntax mistakes. They further argue that many failures arise at integration and orchestration boundaries. This directly supports the premise of the current study. In mature TypeScript ecosystems, fragility is often located in the surrounding toolchain rather than in local program logic.

These two studies are important, but they stop short of the present research question. Bogner and Merkel compare language populations at repository scale, while Tang et al. characterize fault patterns across the TypeScript ecosystem. Neither paper examines whether using one shared resolver, one transform pipeline, and one CLI entry point across packages changes test consistency inside a single full-stack monorepo.

B. Continuous Integration and Automated Testing Infrastructure

Wang et al. study test automation maturity in CI contexts and report that higher maturity is associated with better product quality and shorter release cycles [3]. Their results show that reliable automated testing infrastructure matters to project outcomes. However, the unit of analysis is the CI practice of a project as a whole, not the consistency of test environments across packages within one repository.

Yu et al. examine automated non-functional requirement testing in CI environments through a multi-case industry study [4]. Their findings describe CI as an environment composed of tools, components, metrics, and feedback mechanisms that enable and support different forms of testing. This is useful for the present paper because it emphasizes that test behavior depends on infrastructure, not only on test cases. At the same time, the study is concerned with NFR evaluation in CI environments rather than with per-package differences in local test stacks.

The CI literature therefore establishes that automated testing quality depends on the surrounding environment and that shared infrastructure can improve evaluation and feedback. What it does not isolate is the narrower problem examined here: the same monorepo may still run different test runners, transforms, and resolution rules in different packages even when all tests are automated in CI.

In summary, the reviewed literature shows three things. First, TypeScript improves some code-level quality indicators but does not by itself eliminate bug-related problems [1]. Second, current TypeScript failures often cluster around configuration, integration, and tooling boundaries [2]. Third, CI research shows that testing outcomes depend heavily on automation infrastructure [3]; [4]. The remaining gap is the

effect of toolchain unification on testing consistency across packages in a TypeScript monorepo. That gap motivates the Circles case study.

III. SYSTEM DESCRIPTION

IV. EVALUATION

V. DISCUSSION

VI. CONCLUSION

ACKNOWLEDGMENT

REFERENCES

- [1] J. Bogner and M. Merkel, “To Type or Not to Type? A Systematic Comparison of the Software Quality of JavaScript and TypeScript Applications on GitHub,” in *Proceedings of the 19th International Conference on Mining Software Repositories*, ACM, 2022, pp. 658–669. doi: 10.1145/3524842.3528454.
- [2] T. Tang, S. Alimadadi, and N. Sumner, “From Logic to Toolchains: An Empirical Study of Bugs in the TypeScript Ecosystem,” in *Proceedings of the 23rd International Conference on Mining Software Repositories*, ACM, 2026, pp. 1–11. doi: 10.1145/3793302.3793379.
- [3] Y. Wang, M. V. Mäntylä, Z. Liu, and J. Markkula, “Test Automation Maturity Improves Product Quality: Quantitative Study of Open Source Projects Using Continuous Integration,” *Journal of Systems and Software*, vol. 188, p. 111259, 2022, doi: 10.1016/j.jss.2022.111259.
- [4] L. Yu, E. Alégroth, P. Chatzipetrou, and T. Gorschek, “Automated NFR Testing in Continuous Integration Environments: A Multi-Case Study of Nordic Companies,” *Empirical Software Engineering*, vol. 28, no. 144, 2023, doi: 10.1007/s10664-023-10356-1.